

Synthetic Dataset Generation Plan

Cloud Resource Governance — 18 Hand-Crafted Scenarios for Multi-Tier Telemetry Synthesis

Document type	Standalone data-generation specification
Version	v3 — May 24, 2026
Scope	Synthetic telemetry generation only. Independent of the agent system that consumes it.
Generation method	Two-pass LLM synthesis + scenario-metadata sidecar + consumer-side read contract
Scenarios	18 hand-crafted, reverse-engineered from target recommendations
Sampling	15-minute intervals × 14 days = 1,344 records per tier
Tiers	Compute, Database, Network, Cache (where relevant)
Intended use	Hand-off to code generation / Codex for synthesis pipeline + consumer-side data access

What's new in v3

v3 adds a **consumer-side data access contract** that specifies the canonical read operations against the bundle (telemetry + sidecar). This closes a gap surfaced during the agent-system design pass: the dataset doc defined what data exists but not how downstream consumers access it. The new contract makes the read surface explicit and future-proofs the dataset for any consumer, not just the agent system being built alongside it.

- **Consumer-side data access contract (new §2.6):** canonical read operations on telemetry and sidecar — telemetry queries (time-series, summary statistics, time-pattern, threshold-breach detection), sidecar queries (configuration, SLA, cost, top-N fixtures, per-instance breakouts, before/after evidence), and cross-tier queries (correlation evidence, for synthesis-layer consumers only).
- **Bundle layout (new §2.7):** each scenario's deliverable is a bundle (telemetry JSON + sidecar). The read contract operates against the bundle as a unit.
- **Tightened recommendation-sourcing principle (§1.7):** now references the read contract so downstream agents have an unambiguous path from "what the recommendation needs" to "which read operation produces it."
- **New QA check (§8.1):** validates that sidecar fields referenced by the read contract are present and non-empty for any scenario that uses them.

Carried forward from v2

- Scenario Metadata sidecar (non-time-varying data — instance classes, counts, replica counts, monthly costs, top-N fixtures).
- Extended telemetry schema: application_p95_latency_ms in Compute_Metrics and db_query_p95_latency_ms in Database_Metrics.

- Inlined before/after evidence in Pass 2 Key Evidence for every scenario.
- Healthy Baselines appendix with explicit numeric ranges per tier.
- Quantitative metric ranges across all 18 scenarios (no qualitative-only scenarios remain).
- Recommendation-sourcing principle (§1.7), weekday/weekend convention (§1.8), Pass 2 baseline reference (§1.9).

What this document is

A complete, self-contained specification for generating the synthetic telemetry dataset that exercises a multi-tier cloud resource governance system. It describes the two-pass generation process, the per-tier JSON schema, the scenario metadata sidecar, sampling parameters, all 18 scenarios in full, ready-to-use LLM prompts for both passes, and a quality-assurance section for verifying generated output.

What this document is not

Not an architecture document for the agent system that will consume this data. The agent system is described separately. This document treats the dataset purely as a deliverable.

1. Purpose and Design Principles

1.1 Why a synthetic dataset

The system being built reasons over multi-tier cloud telemetry to produce balanced optimization recommendations across cost, performance, and reliability. Real cloud telemetry is unreliable for evaluation: it does not deterministically exercise the scenarios we want the system to handle, and ground-truth recommendations cannot be established without months of observation and human review. A hand-crafted synthetic dataset solves both problems. Each scenario has a known target recommendation; the telemetry plus sidecar metadata is reverse-engineered to be the evidence that justifies that recommendation.

1.2 Reverse-engineering principle

Every scenario in this document was constructed in this order: (1) a target rich recommendation was hand-written, (2) the business context, SLA, current configuration, and sidecar metadata were chosen to make that recommendation appropriate, (3) the metric ranges, time patterns, and (where relevant) cross-tier correlations were specified to be the evidence trail that leads naturally to the recommendation. The dataset's job is to make each target recommendation a reasoned conclusion, not a leap.

1.3 Why two passes

Generation is split into two passes for a practical reason, not an architectural one. No amount of prompt engineering reliably produces, in a single LLM generation, both (a) clean per-tier base time-series with realistic daily patterns and (b) correct, tightly-coupled cross-tier correlations. The model is competent at either task in isolation but conflates them when asked to do both at once.

Pass 1 produces the independent base time-series for each tier. Pass 2 takes the exact Pass 1 output and surgically adds cross-tier correlations for the scenarios that require them, while preserving every Pass 1 signal without modification. Scenarios that do not require correlation pass through Pass 2 unchanged.

1.4 Sampling parameters

All time-series use **15-minute intervals over 14 full days**, matching common AWS CloudWatch granularity.

Calculation	Value
Intervals per hour	4
Hours per day	24
Records per day per tier	96
Days	14 (2 weeks, starting Monday 2026-05-01)
Total records per tier	1,344
Timestamp format	ISO 8601 UTC, e.g. 2026-05-01T00:00:00Z

1.5 The "11 of 14 days" rule

Where a scenario specifies a recurring pattern, the pattern must hold on **at least 11 of the 14 days**. This threshold is high enough that the pattern is statistically unambiguous while low enough to leave room for realistic day-to-day variation.

1.6 The before/after evidence device

Many rich recommendations include specific numbers — instance sizes, percentage savings, dollar amounts, latency reductions. These numbers cannot be derived from the telemetry alone. To allow the generator to produce them without hallucination, scenarios include a **before/after evidence** clause of the form: *"In other apps with identical configuration X, after change Y, outcome Z was observed."* In v2 this clause is inlined into the Pass 2 Key Evidence field for tight coupling.

1.7 Recommendation-sourcing principle (updated in v3)

An adversarial review of v1 found that recommendations regularly reference specific values — instance counts, replica counts, top-N queries or cache keys, dollar amounts — that the telemetry schema cannot represent. v2 introduced the sidecar to carry these. v3 makes the sourcing explicit by tying each recommendation component to the read operation that produces it (defined in §2.6).

Recommendation component	Sourced from	Read operation (§2.6)
Whether the scenario applies	Telemetry (Pass 1 + Pass 2)	get_summary_statistics, get_time_pattern, detect_threshold_breaches
Cross-tier cause-effect	Pass 2 correlation spec	get_correlation_evidence
Specific instance classes (from/to)	Sidecar current_configuration + before/after evidence	get_configuration, get_before_after_evidence
Specific replica/node/pool counts	Sidecar + before/after evidence	get_configuration, get_before_after_evidence
Dollar amounts and percentage savings	Sidecar monthly_cost_usd + before/after evidence	get_monthly_cost, get_before_after_evidence
Top-N queries or cache keys	Sidecar fixtures	get_top_queries, get_top_cache_keys
Per-instance variance details (Scenario 5)	Sidecar per_instance_imbalance	get_per_instance_breakout
Target outcome metrics	Before/after evidence in Pass 2 Key Evidence	get_before_after_evidence

The telemetry establishes that a scenario applies. The sidecar metadata and before/after evidence provide the specific quantities. The read contract in §2.6 defines the access surface for both.

1.8 Weekday/weekend convention (new in v2)

The 14-day window starts on a Monday (2026-05-01). Weekdays are days 1–5 and 8–12; weekends are days 6–7 and 13–14. Unless a scenario explicitly states otherwise:

- Business-hours patterns (e.g. "spikes during 10–11 AM") occur on weekdays only.
- Weekend traffic is approximately 40–60% of weekday baseline for production-facing services.
- Internal/analytics services may show near-zero weekend activity (10–25% of weekday baseline).
- Healthy services maintain their normal range across both weekdays and weekends; "healthy" never means "zero activity."
- The "11 of 14" rule for recurring patterns is naturally satisfied by weekday-only patterns (10 weekdays = 10 of 14; the rule allows ≥ 11 , so weekday-only patterns must show on all 10 weekdays plus at least one weekend day, or the scenario must explicitly state weekday-only).

Note on the 11-of-14 rule for weekday-only patterns: a strict weekday-only pattern appears on 10 days. For these scenarios, the scenario specification will say "weekday-only" and the QA validator should require the pattern on all 10 weekdays, treating weekends as explicitly exempt from the 11-of-14 count.

1.9 Pass 2 baseline reference (new in v2)

When Pass 2 correlation specifications use relative magnitudes (e.g. "database CPU increases by 38–45%", "connections increase 3x"), these are interpreted as adjustments to the scenario's Pass 1 baseline range, not as absolute values:

Example. If Pass 1 establishes database CPU baseline of 35–45% during business hours, a Pass 2 instruction "DB CPU increases by 38–45%" means absolute CPU rises to approximately 48–65% (baseline + relative bump) during the correlation window — not absolute CPU of 38–45%. Generators must compute the absolute target from the baseline.

2. JSON Schema — Telemetry and Sidecar

2.1 Top-level structure

Each scenario produces a single JSON object with two top-level concerns: the **telemetry** (up to four tier arrays, 1,344 records each) and the **scenario_metadata** sidecar (non-time-varying data the recommendation references).

2.2 Telemetry — per-tier field schemas

v2 extends Compute_Metrics and Database_Metrics with two latency fields that were referenced by Pass 2 correlations in v1 but had no schema home.

Compute_Metrics

Field	Type	Notes
timestamp	string (ISO 8601 UTC)	e.g. "2026-05-01T00:00:00Z"
cpu_p50	float	Median CPU utilization %
cpu_p90	float	90th percentile CPU utilization %
cpu_p95	float	95th percentile CPU utilization %
memory_p50	float	Median memory utilization %
memory_p90	float	90th percentile memory utilization %
application_p95_latency_ms	float (new in v2)	95th percentile end-to-end application response latency

Database_Metrics

Field	Type	Notes
timestamp	string (ISO 8601 UTC)	Aligned to Compute_Metrics timestamps
cpu_p90	float	90th percentile CPU utilization %
connections_p95	integer	95th percentile concurrent connections
slow_query_percent	float	Percentage of queries exceeding the slow-query threshold
db_query_p95_latency_ms	float (new in v2)	95th percentile database query latency

Network_Metrics

Field	Type	Notes
timestamp	string (ISO 8601 UTC)	Aligned to other tier timestamps
p95_latency_ms	float	95th percentile inter-service / network-layer latency
error_rate	float	Network/request error rate as decimal (e.g. 0.012 = 1.2%)

Cache_Metrics (when relevant)

Field	Type	Notes
timestamp	string (ISO 8601 UTC)	Aligned to other tier timestamps

Field	Type	Notes
hit_ratio	float	Cache hit ratio as percentage (e.g. 71.2 = 71.2%)

2.3 Scenario Metadata sidecar (new in v2)

The sidecar carries everything the recommendation references that is not a time-series observation. It is generated **statically per scenario** (no LLM time-series synthesis needed for this block — it is authored as part of the scenario specification and emitted verbatim alongside the telemetry).

Field	Type	Notes
scenario_id	integer	1–18
scenario_name	string	Human-readable scenario name
scenario_type	string	Single-tier / Cross-tier / Mixed / Healthy / Mostly Healthy
business_context	string	1-line description of the application
sla_target	string	e.g. "99.9%"
current_configuration	object	See sub-schema below — captures instance classes, counts, pool sizes, replica counts
monthly_cost_usd	number	Approximate current monthly spend; used to ground dollar-amount recommendations
top_queries	array (optional)	For scenarios 4, 8 — list of slow query identifiers and their average latency_ms
top_cache_keys	array (optional)	For scenario 7 — list of cache key patterns by frequency
before_after_outcomes	array	List of {config, change, outcome} records used to ground specific recommendation quantities

current_configuration sub-schema

Field	Type	Notes
compute_instance_class	string (optional)	e.g. "t3.large"
compute_instance_count	integer (optional)	Current number of compute instances
compute_min_replicas	integer (optional)	Minimum replica count if scaling is configured
compute_max_replicas	integer (optional)	Maximum replica count if scaling is configured
compute_scaling_policy	string (optional)	e.g. "none", "basic", "predictive", "scheduled"
load_balancer_algorithm	string (optional)	e.g. "round_robin", "least_outstanding_requests"
database_instance_class	string (optional)	e.g. "db.r6g.xlarge"
database_read_replicas	integer (optional)	Number of read replicas (0 if none)
database_connection_pool_limit	integer (optional)	Current pool size limit
cache_node_count	integer (optional)	Number of cache cluster nodes
network_topology	string (optional)	e.g. "basic_vpc_peering", "private_link"

2.4 Full output structure example

```
{
  "scenario_id": 4,
  "scenario_name": "Database Connection Bottleneck",
  "pass": 2,
  "scenario_metadata": {
    "scenario_type": "Single-tier (Negative)",
    "business_context": "High-criticality user service",
    "sla_target": "99.9%",
    "current_configuration": {
      "database_instance_class": "db.r6g.large",
      "database_read_replicas": 0,
      "database_connection_pool_limit": 100
    },
    "monthly_cost_usd": 4200,
    "top_queries": [
      {"query_id": "q_user_lookup_by_email",
       "avg_latency_ms": 820, "calls_per_hour": 14000},
      {"query_id": "q_session_validation",
       "avg_latency_ms": 640, "calls_per_hour": 22000},
      {"query_id": "q_profile_aggregate",
       "avg_latency_ms": 510, "calls_per_hour": 3200},
      {"query_id": "q_audit_log_join",
       "avg_latency_ms": 480, "calls_per_hour": 800},
      {"query_id": "q_permissions_recursive",
       "avg_latency_ms": 410, "calls_per_hour": 1900}
    ],
    "before_after_outcomes": [
      {"config": "single db.r6g.large, no replicas, pool=100",
       "change": "optimize top 5 queries + pool=200 + 2 read replicas",
       "outcome": "slow_query_percent: 8% -> <2%, SLA preserved"}
    ]
  },
  "Compute_Metrics": [ /* 1,344 records */ ],
  "Database_Metrics": [ /* 1,344 records */ ],
  "Network_Metrics": [],
  "Cache_Metrics": []
}
```

2.5 Output rules (apply to both passes)

- Use ISO 8601 timestamps in UTC, aligned across all tier arrays in the same scenario.
- Use realistic floating-point values that match the ranges and time patterns specified in the scenario.
- Output only the JSON object — no explanatory prose, no markdown fencing.
- For tiers not used by a scenario, output an empty array [] under that tier key.
- Every tier array used must contain exactly 1,344 records.
- The scenario_metadata block is authored statically per scenario and emitted verbatim by both passes.
- Floating-point values use one decimal place where reasonable; more for cache hit ratio or error rate.

2.6 Consumer-side data access contract (new in v3)

The dataset is consumed by downstream agents and analysis tools. To keep the dataset future-proof and to make the recommendation-sourcing principle (§1.7) concrete, v3 defines a **canonical read contract** that any consumer should implement against the bundle (telemetry + sidecar). These are abstract operations — consumers may implement them as Python functions, SQL queries, REST endpoints, or any other surface —

but the semantics are fixed.

Defining the read contract in the dataset doc (rather than only in the consumer's doc) ensures: (1) the sidecar fields are authored knowing how they will be queried, (2) multiple consumers can be built against the same contract without diverging, and (3) the audit trail in any consumer can log read operations against a stable vocabulary.

Telemetry read operations

Operation	Signature	Returns
get_time_series	(tier, window_start, window_end)	List of records for the tier within the window. Default window = full 14 days.
get_summary_statistics	(tier, metric, window)	Dict with p50, p90, p95, min, max, mean, stddev for the metric over the window.
get_time_pattern	(tier, metric, granularity)	Daily and weekday/weekend patterns at the requested granularity (hour, 15min).
detect_threshold_breaches	(tier, metric, threshold, comparator)	List of (timestamp, value) tuples where the metric crosses the threshold.
get_metric_distribution	(tier, metric, window)	Histogram-shaped distribution for the metric over the window.

Sidecar read operations

Operation	Signature	Returns
get_business_context	()	Business context string from the sidecar.
get_sla_target	()	SLA target string (e.g. "99.9%").
get_configuration	(tier)	Tier-scoped configuration dict from current_configuration (instance class, count, scaling policy, etc.).
get_monthly_cost	()	Approximate current monthly USD spend (number).
get_top_queries	()	Array of slow-query fixtures (Scenarios 4 and 8) — query_id, avg_latency_ms, calls_per_hour.
get_top_cache_keys	()	Array of high-frequency cache key fixtures (Scenario 7) — key_pattern, traffic_share.
get_per_instance_breakout	()	Per-instance variance description (Scenario 5) — hot/cold instance counts and CPU bands.
get_before_after_evidence	()	Before/after outcome record(s) — config, change, outcome — for grounding specific recommendation quantities.

Cross-tier read operations (synthesis layer only)

Operation	Signature	Returns
get_correlation_evidence	(tier_a, metric_a, tier_b, metric_b, window)	Time-aligned signal pairs and correlation statistics across two tiers. Intended for the cross-tier evaluator / synthesis layer; individual tier specialists should not use this.
get_pattern_alignment	(tier_a, tier_b, granularity)	Confirms whether two tiers exhibit aligned patterns (e.g. simultaneous business-hours peaks).

Access discipline. Tier-specialist consumers should restrict themselves to telemetry and sidecar operations scoped to their own tier. Cross-tier operations are reserved for synthesis-layer consumers. This discipline mirrors the architectural separation between independent per-tier analysis and explicit cross-tier synthesis, and it makes specialist reasoning traces honest — a specialist's conclusion cannot rest on evidence it was structurally prevented from accessing.

2.7 Bundle layout (new in v3)

Each scenario's deliverable is a **bundle** — a single JSON document containing both the telemetry tier arrays and the `scenario_metadata` sidecar. The read contract in §2.6 operates against the bundle as a unit. The bundle is the dataset's deliverable artifact; Pass 1 and Pass 2 outputs are intermediate artifacts in the generation pipeline.

```
{
  "scenario_id": 7,
  "scenario_name": "Cache Miss Cascade",
  "pass": 2,
  "bundle_version": "v3",
  "scenario_metadata": { /* sidecar fields */ },
  "Compute_Metrics": [ /* 1,344 records */ ],
  "Database_Metrics": [ /* 1,344 records */ ],
  "Network_Metrics": [],
  "Cache_Metrics": [ /* 1,344 records */ ]
}
```

A consumer implementing the read contract receives this bundle as input and exposes the operations defined in §2.6 against it. Implementations are deliberately not specified here — that is the consumer's choice.

3. Two-Pass Generation Process

3.1 Pass 1 — Base time-series per tier

Pass 1 generates the independent base time-series for every tier required by a scenario. The LLM is given the scenario's business context, SLA, current configuration, key metric ranges (including healthy-baseline ranges for tiers marked healthy), and time pattern. It is not given any cross-tier correlation instructions, even for scenarios that ultimately need them. This isolation is what makes Pass 1 reliable.

3.2 Pass 2 — Correlation injection and evidence inlining

Pass 2 takes the exact JSON output from Pass 1 and modifies it according to the scenario's Pass 2 specification. The contract:

- Every Pass 1 signal is preserved exactly. Pass 2 layers correlation effects on top; it does not redo the base data.
- For scenarios marked "Cross-Tier Correlation: None," the time-series output matches Pass 1 bit-for-bit.
- For scenarios with a Cross-Tier Correlation specification, the correlated tier's values are adjusted in the time windows where the source signal exceeds its threshold. Adjustments are applied within the stated timing window (e.g. "same 15-min window" or "within 15–30 minutes").
- Correlation magnitudes (e.g. "+38–45%", "3x") are interpreted relative to the Pass 1 baseline (section 1.9).
- Pattern frequency holds on at least 11 of 14 days, or all 10 weekdays for weekday-only patterns (section 1.8).
- Scenario metadata sidecar is emitted unchanged.
- Key Evidence in the scenario specification carries the inlined before/after observation from identical configurations — this is reference content for downstream consumers, not telemetry modification.

3.3 Pipeline summary

Step	Input	Output
Author scenario	Section 6 scenario specification	scenario_spec.yaml (Pass 1 + Pass 2 fields + sidecar)
Pass 1	scenario_spec (Pass 1 fields + sidecar)	Per-tier JSON time-series with base patterns + sidecar
Pass 2	Pass 1 JSON + Pass 2 spec	Final JSON with correlations injected + sidecar passed through
QA validate	Final JSON	Pass/fail report
Bundle	Validated JSON	Scenario package for downstream consumption

4. Scenario Catalog — Summary Table

All 18 scenarios at a glance. Detailed specifications follow in section 6.

#	Scenario Name	Compute	Data Layer	Network	Type	Recommended Solution
1	Chronic Underutilization	Low	—	—	Single-tier	Downsize compute instances
2	Spiky Compute Load	High spikes	—	—	Single-tier	Implement auto-scaling
3	Over-provisioned Database	—	Over-provisioned	—	Single-tier	Downsize database instance
4	Database Connection Bottleneck	—	High connections	—	Single-tier	Optimize queries + increase pool
5	Load Balancer Inefficiency	—	—	Uneven load	Single-tier	Reconfigure load balancer
6	Healthy Application	Healthy	Healthy	Healthy	Healthy	No change required
7	Cache Miss Cascade	High load	High load	—	Cross-tier	Improve cache strategy
8	Database Bottleneck Impact	High latency	Slow queries	—	Cross-tier	Optimize slow database queries
9	Peak Hours Cost vs Reliability	Over-provisioned	Over-provisioned	Over-provisioned	Cross-tier	Scheduled scaling
10	Network Latency Impact	High latency	—	High latency	Cross-tier	Optimize service communication
11	Multi-Tier Over-provisioning	Over-provisioned	Over-provisioned	Over-provisioned	Cross-tier	Right-size all tiers
12	Healthy Compute, Problematic Database	Healthy	Over-provisioned	—	Mixed	Downsize database only
13	Compute Spike + Database Strain	High spikes	High connections	—	Cross-tier	Scale compute + optimize DB
14	Good Performance, High Cost	Healthy	Over-provisioned	Healthy	Mixed	Right-size compute and database
15	Reliability Focused Over-provisioning	Healthy	Healthy	Over-provisioned	Mixed	Review SLA necessity
16	Partial Optimization	Low utilization	Healthy	Healthy	Single-tier	Minor compute adjustment
17	Cross-Tier Performance Degradation	High latency	High latency	High latency	Cross-tier	Full end-to-end trace analysis
18	Mostly Healthy with Minor Inefficiency	Minor low util	Healthy	Healthy	Mostly healthy	Minor compute adjustment

5. Scenario Taxonomy, Evaluation Properties, and Healthy Baselines

5.1 Evaluation properties

Every scenario is tagged with one or more evaluation properties — the analytical capabilities the scenario is designed to exercise.

Property	What it tests
Single-tier detection	Identification of a problem confined to one tier without false attribution.
Cross-tier correlation reading	Recognition of cause-effect relationships between tiers from time-aligned signals.
Trade-off reasoning	Balancing cost, performance, and reliability when they pull in different directions.
SLA-aware reasoning	Recommendations that respect the application's stated availability target.
Restraint	Producing "no action" or "minor adjustment only" when the data does not justify aggressive change.
Diagnostic deferral	Recommending further investigation rather than action when root cause is ambiguous.
Specificity grounding	Producing recommendations with specific quantities backed by sidecar metadata and before/after evidence.
Mixed-state handling	Correctly handling applications where some tiers are healthy and others are not.
Time-pattern recognition	Detection of bimodal, peak-hours, or business-hours patterns.
Business-context questioning	Recognizing when the right recommendation is to question business inputs (e.g. SLA target) rather than re-architect.

5.2 Healthy Baselines (new in v2)

Several scenarios (6, 12, 14, 15, 16, 18) mark one or more tiers as *healthy*. To remove ambiguity, the following baseline ranges define what "healthy" means per tier. When a scenario tags a tier as healthy, the generator uses these ranges unless the scenario specifies otherwise.

Healthy Compute_Metrics ranges

Field	Healthy range	Notes
cpu_p50	40–60%	Sweet spot for production utilization
cpu_p90	55–72%	Headroom remains for normal spikes
cpu_p95	62–78%	Below scaling threshold
memory_p50	45–60%	Comfortable working set
memory_p90	55–72%	Headroom for caching and bursts
application_p95_latency_ms	120–220	Within tight SLA envelopes

Healthy Database_Metrics ranges

Field	Healthy range	Notes
cpu_p90	35–55%	Active but not saturated
connections_p95	40–90	Below typical pool limits

Field	Healthy range	Notes
slow_query_percent	0.5–2.5%	Background noise, not a problem signal
db_query_p95_latency_ms	30–120	Indexed queries; no table scans

Healthy Network_Metrics ranges

Field	Healthy range	Notes
p95_latency_ms	25–80	Same-region inter-service traffic
error_rate	0.001–0.01 (0.1–1.0%)	Background errors only

Healthy Cache_Metrics ranges

Field	Healthy range	Notes
hit_ratio	88–96%	Well-warmed cache with appropriate key design

6. Full Scenario Specifications

Each scenario is presented with its Pass 1 base specification, Pass 2 modification, scenario metadata sidecar contents, and evaluation property tags. v2 strengthens Pass 2 Key Evidence by inlining the before/after observation from identical configurations.

Scenario 1: Chronic Underutilization

Type: Single-tier (Negative)

Tiers used: Compute_Metrics only.

Evaluation properties: *Single-tier detection · Specificity grounding · Cost-axis reasoning*

Pass 1 — Base Data

Business Context & SLA: Medium-criticality internal service. SLA = 99.5% availability.

Current Configuration: Fixed t3.large instances, 8 replicas, no auto-scaling.

Key Metrics: CPU p50 12–22%, p90 16–26%, p95 18–28%. Memory p50 15–25%, p90 20–32%. application_p95_latency_ms 80–140 (well below any SLA-related threshold).

Time Pattern: Extremely flat across all 14 days, no business-hours variation.

Cross-Tier Correlation: None.

Key Evidence: Consistently low utilization with no spikes.

Target Recommendation: Switch compute instances from t3.large to t3.medium and reduce minimum replica count from 8 to 4. Expected savings: 62% on monthly compute cost (~\$2,850) with no impact on SLA.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Consistently low utilization with no spikes. In other apps with identical t3.large × 8 fixed configuration, after switching to t3.medium × 4 with no auto-scaling, CPU p50 stayed in the 12–22% range and SLA remained at 99.5% while monthly compute cost dropped 62%.

Scenario Metadata Sidecar

Compute Instance Class: t3.large

Compute Instance Count: 8

Compute Scaling Policy: none

Monthly Cost Usd: 4600

Before After: config: t3.large × 8 fixed → change: t3.medium × 4 fixed → outcome: CPU p50 stayed 12–22%, SLA 99.5%, monthly cost dropped 62% (~\$2,850 saved)

Scenario 2: Spiky Compute Load

Type: Single-tier (Negative)

Tiers used: Compute_Metrics only.

Evaluation properties: Single-tier detection · Time-pattern recognition · Specificity grounding

Pass 1 — Base Data

Business Context & SLA: High-criticality checkout service. SLA = 99.9% availability.

Current Configuration: Fixed capacity, 6 m5.large instances, no auto-scaling.

Key Metrics: Baseline CPU p50 25–35%, p90 30–42%. During spikes (10:00–11:00 and 15:00–16:00 weekdays): CPU p95 88–96%. Memory p50 35–48%, p90 45–60%. application_p95_latency_ms 180–260 baseline, rising to 420–580 during spikes.

Time Pattern: Weekday-only. Two predictable daily spikes (10:00–11:00 and 15:00–16:00). Weekends show flat baseline only.

Cross-Tier Correlation: None.

Key Evidence: Large gap between p50 and p95 with consistent daily timing on weekdays.

Target Recommendation: Implement predictive auto-scaling with scheduled capacity increases during 10–11 AM and 3–4 PM windows.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Large gap between p50 and p95 with consistent daily timing. In other apps with identical fixed-capacity configuration, after implementing predictive auto-scaling, p95 spikes dropped below 65% while maintaining 99.9% SLA and reducing baseline cost by 48%.

Scenario Metadata Sidecar

Compute Instance Class: m5.large

Compute Instance Count: 6

Compute Scaling Policy: none

Monthly Cost Usd: 5200

Before After: config: m5.large × 6 fixed → change: predictive scheduled auto-scaling 10–11AM and 3–4PM → outcome: p95 dropped below 65%, SLA 99.9% preserved, baseline cost down 48%

Scenario 3: Over-provisioned Database

Type: Single-tier (Negative)

Tiers used: Database_Metrics only.

Evaluation properties: Single-tier detection · Specificity grounding · Cost-axis reasoning

Pass 1 — Base Data

Business Context & SLA: Medium-criticality analytics backend. SLA = 99.7% availability.

Current Configuration: db.r6g.xlarge instance, 0 read replicas, manual storage management.

Key Metrics: Database CPU p90 < 24%, connections_p95 18–35, slow_query_percent 0.5–1.5%, db_query_p95_latency_ms 40–95.

Time Pattern: Flat and low utilization all day, weekdays and weekends.

Cross-Tier Correlation: None.

Key Evidence: Large instance class with extremely low usage across all 14 days.

Target Recommendation: Downsize primary RDS instance from db.r6g.xlarge to db.r6g.large and enable storage auto-scaling. Expected savings: \$1,680 per month.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Large instance class with extremely low usage. In other apps with identical db.r6g.xlarge configuration, after downsizing to db.r6g.large with storage auto-scaling enabled, CPU p90 stayed below 24% and SLA remained at 99.7% with \$1,680 monthly savings.

Scenario Metadata Sidecar

Database Instance Class: db.r6g.xlarge

Database Read Replicas: 0

Monthly Cost Usd: 3360

Before After: config: db.r6g.xlarge with manual storage → change: db.r6g.large with storage auto-scaling → outcome: CPU p90 stayed below 24%, SLA 99.7%, \$1,680/month savings

Scenario 4: Database Connection Bottleneck

Type: Single-tier (Negative)

Tiers used: Database_Metrics only.

Evaluation properties: *Single-tier detection · Performance-axis reasoning · Specificity grounding*

Pass 1 — Base Data

Business Context & SLA: High-criticality user service. SLA = 99.9% availability.

Current Configuration: Single db.r6g.large instance, 0 read replicas, connection pool limit = 100.

Key Metrics: Connection count p95 > 145 during business hours (overflow above pool), slow_query_percent > 8% during business hours (2–3% otherwise). cpu_p90 55–70%. db_query_p95_latency_ms 350–650 during business hours, 80–140 otherwise.

Time Pattern: Weekday business-hours peaks (9 AM – 6 PM). Weekends show baseline only.

Cross-Tier Correlation: None.

Key Evidence: High connection count correlating with increased slow queries during business hours; pool limit (100) is exceeded.

Target Recommendation: Optimize the top 5 slowest queries (see sidecar top_queries fixture), increase database connection pool limit from 100 to 200, and add read replica offloading.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): High connection count correlating with increased slow queries. In other apps with identical single-large-instance configuration, after query optimization, pool increase to 200, and read replica offloading, `slow_query_percent` dropped below 2% and SLA stayed at 99.9%.

Scenario Metadata Sidecar

Database Instance Class: db.r6g.large

Database Read Replicas: 0

Database Connection Pool Limit: 100

Monthly Cost Usd: 2100

Top Queries: 5 queries identified with `avg_latency_ms` 410–820, primarily covering user lookup, session validation, profile aggregation, audit log joins, and recursive permissions checks

Before After: config: db.r6g.large single, pool=100, 0 replicas → change: optimize top 5 queries + pool=200 + 2 read replicas → outcome: `slow_query_percent` dropped below 2%, SLA 99.9%

Scenario 5: Load Balancer Inefficiency

Type: Single-tier (Negative)

Tiers used: Compute_Metrics (cluster-level aggregates show wide p50–p95 spread reflecting per-instance imbalance; sidecar carries explicit per-instance figures).

Evaluation properties: *Single-tier detection · Distribution analysis*

Pass 1 — Base Data

Business Context & SLA: High-criticality API gateway. SLA = 99.9% availability.

Current Configuration: 8 c5.xlarge instances behind an ALB with round-robin target group algorithm.

Key Metrics: Cluster CPU p50 30–40%, p90 75–85%, p95 88–94% — wide spread reflects imbalanced load. Memory p50 35–45%, p90 50–65%. `application_p95_latency_ms` 220–340.

Time Pattern: Persistent imbalance across all 14 days, weekdays and weekends similarly.

Cross-Tier Correlation: None.

Key Evidence: Sustained gap between p50 and p95 over the entire window indicates persistent per-instance variance, not transient spikes. Sidecar confirms 4 instances at >85% CPU and 4 at <15% CPU on a rolling-hour basis.

Target Recommendation: Reconfigure target group from round-robin to least-outstanding-requests algorithm. Expected outcome: CPU variance across instances drops below 12%.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): High variance in instance utilization. In other apps with identical round-robin configuration, after switching to least-outstanding-requests, CPU variance dropped below 12% with no SLA impact.

Scenario Metadata Sidecar

Compute Instance Class: c5.xlarge

Compute Instance Count: 8

Load Balancer Algorithm: round_robin

Monthly Cost Usd: 3800

Per Instance Imbalance: 4 instances consistently above 85% CPU (hot), 4 consistently below 15% CPU (cold)
— measured rolling-hourly across 14 days

Before After: config: 8 × c5.xlarge with round-robin ALB → change: least-outstanding-requests target group →
outcome: CPU variance dropped below 12%, SLA 99.9% preserved

Scenario 6: Healthy Application

Type: Healthy

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics, Cache_Metrics.

Evaluation properties: Restraint · Mixed-state handling (positive case)

Pass 1 — Base Data

Business Context & SLA: Medium-criticality internal tool. SLA = 99.5% availability.

Current Configuration: All tiers right-sized: 4 t3.medium compute instances with predictive auto-scaling (min 4 / max 8), db.r6g.large database with 1 read replica, 4-node cache cluster.

Key Metrics: All tiers use the Healthy Baselines from section 5.2. Compute: CPU p50 45–60%, p95 68–78%, memory p50 45–60%, application_p95_latency_ms 140–200. Database: cpu_p90 38–50%, connections_p95 50–75, slow_query_percent 0.8–2.0%, db_query_p95_latency_ms 45–90. Network: p95_latency_ms 30–70, error_rate 0.002–0.008. Cache: hit_ratio 88–94%.

Time Pattern: Normal daily pattern with moderate weekday peaks; weekend traffic ~50% of weekday baseline.

Cross-Tier Correlation: None.

Key Evidence: All metrics in healthy operating range across all 14 days. No threshold breaches, no SLA pressure.

Target Recommendation: All tiers are correctly sized and balanced. No action required.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): All metrics in healthy operating range. In other apps with identical configuration, no change was needed and metrics and SLA were sustained over the observation window.

Scenario Metadata Sidecar

Compute Instance Class: t3.medium

Compute Instance Count: 4

Compute Scaling Policy: predictive (min 4, max 8)

Database Instance Class: db.r6g.large

Database Read Replicas: 1

Cache Node Count: 4

Monthly Cost Usd: 3900

Before After: Reference baseline. No change recommended.

Scenario 7: Cache Miss Cascade

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Database_Metrics, Cache_Metrics.

Evaluation properties: *Cross-tier correlation reading · Performance-axis reasoning · Specificity grounding*

Pass 1 — Base Data

Business Context & SLA: High-criticality recommendation engine. SLA = 99.9% availability.

Current Configuration: Cache cluster: 3 nodes. Compute: 6 m5.large. Database: db.r6g.large with 1 read replica.

Key Metrics: Cache hit_ratio 58–72%. Compute baseline cpu_p50 35–48%, application_p95_latency_ms baseline 180–280. Database baseline cpu_p90 35–50%.

Time Pattern: Low cache hit periods occur daily during business hours on weekdays.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: Low cache hit ratio as primary signal.

Target Recommendation: Increase cache cluster from 3 to 6 nodes, implement cache warming logic, and review cache key design for the top 3 most frequent keys (see sidecar). Expected outcome: cache hit ratio improvement from 65% to above 89%.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): When cache hit_ratio drops below 72%, database cpu_p90 rises by 38–45% above its Pass 1 baseline range (35–50% → 48–73%), and application_p95_latency_ms rises by 180–250ms above its baseline (180–280 → 360–530). Effect appears within the same 15-min window. Pattern repeats on at least 11 of 14 days.

Key Evidence (with before/after inlined): Strong negative correlation between cache hit ratio and database CPU and application latency. In other apps with identical 3-node cache configuration, after increasing to 6 nodes plus cache warming, hit ratio rose above 89%, database CPU dropped by 38–45%, and SLA stayed at 99.9%.

Scenario Metadata Sidecar

Cache Node Count: 3

Compute Instance Class: m5.large

Compute Instance Count: 6

Database Instance Class: db.r6g.large

Database Read Replicas: 1

Monthly Cost Usd: 5800

Top Cache Keys: 3 high-frequency keys identified: user_recommendations_by_id (62% of cache traffic), trending_items_by_region (21%), session_personalization_blob (11%)

Before After: config: 3-node cache → change: 6-node cache + warming + key redesign on top 3 → outcome: hit_ratio above 89%, DB CPU down 38–45%, SLA 99.9% preserved

Scenario 8: Database Bottleneck Impact

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Database_Metrics.

Evaluation properties: *Cross-tier correlation reading · Performance-axis reasoning · Specificity grounding*

Pass 1 — Base Data

Business Context & SLA: High-criticality checkout service. SLA = 99.9% availability.

Current Configuration: Database: db.r6g.xlarge, 0 read replicas. Compute: 8 m5.large with basic scaling.

Key Metrics: Database slow_query_percent > 10% during business hours (1.5–3% otherwise).
db_query_p95_latency_ms 450–820 during business hours, 90–180 otherwise. Compute baseline
application_p95_latency_ms 160–200, rising to 580–720 during business hours.

Time Pattern: Weekday business-hours only. Slow queries occur 9 AM – 6 PM.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: Elevated slow query percentage in database during business hours.

Target Recommendation: Optimize the 6 slowest SQL queries (see sidecar) and add 2 read replicas with read/write splitting. Expected outcome: application p95 latency reduction from 650ms to <220ms.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): When db slow_query_percent > 10%, compute application_p95_latency_ms jumps from its Pass 1 baseline (160–200ms) to above 650ms within 15–30 minutes. Pattern repeats on at least 11 of 14 weekday business-hours windows.

Key Evidence (with before/after inlined): Strong time correlation between DB slow queries and application latency. In other apps with identical no-replica configuration, after query optimization and adding 2 read replicas, slow_query_percent dropped below 2% and application p95 latency fell below 220ms while SLA stayed at 99.9%.

Scenario Metadata Sidecar

Database Instance Class: db.r6g.xlarge

Database Read Replicas: 0

Compute Instance Class: m5.large

Compute Instance Count: 8

Monthly Cost Usd: 6400

Top Queries: 6 queries identified covering cart aggregation, inventory checks, pricing waterfall, payment-provider routing, order history, and fulfillment lookup; avg_latency_ms 380–820

Before After: config: db.r6g.xlarge, no replicas → change: optimize top 6 queries + 2 read replicas with R/W split → outcome: slow_query_percent below 2%, app p95 latency below 220ms, SLA 99.9%

Scenario 9: Peak Hours Cost vs Reliability

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Evaluation properties: *Cross-tier correlation reading · Trade-off reasoning · SLA-aware reasoning · Time-pattern recognition*

Pass 1 — Base Data

Business Context & SLA: High-criticality e-commerce platform. SLA = 99.99% availability.

Current Configuration: Fixed capacity, no scheduled scaling. Compute: 20 m5.xlarge. Database: db.r6g.2xlarge with 2 read replicas. Network: cross-AZ private link.

Key Metrics: During peak windows (9–11 AM and 2–4 PM weekdays): compute cpu_p95 85–94%, database cpu_p90 78–88%. Outside peak: compute cpu_p50 12–24%, database cpu_p90 18–28%. application_p95_latency_ms peak 240–320, off-peak 110–160.

Time Pattern: Weekday bimodal pattern only. Weekend baseline ~30% of weekday off-peak.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: Clear peak vs non-peak behavior on weekdays.

Target Recommendation: Implement scheduled scaling policy to reduce capacity by 65% during off-peak hours and weekends while maintaining full capacity during peak windows.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): During peak hours all three tiers show utilization above 85%. Outside peak hours all three tiers drop below 30%. Pattern repeats every weekday on all 10 weekdays (weekday-only, as per section 1.8). Weekends remain in low-utilization state.

Key Evidence (with before/after inlined): Clear bimodal pattern with massive over-provisioning outside peaks. In other apps with identical fixed-capacity configuration, after implementing scheduled scaling, off-peak cost dropped 65% while peak SLA stayed at 99.99%.

Scenario Metadata Sidecar

Compute Instance Class: m5.xlarge

Compute Instance Count: 20

Compute Scaling Policy: none

Database Instance Class: db.r6g.2xlarge

Database Read Replicas: 2

Monthly Cost Usd: 18400

Before After: config: fixed 20 × m5.xlarge → change: scheduled scaling, 65% capacity reduction off-peak and weekends → outcome: off-peak cost down 65%, peak SLA 99.99% preserved

Scenario 10: Network Latency Impact

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Network_Metrics.

Evaluation properties: *Cross-tier correlation reading · Performance-axis reasoning · Specificity grounding*

Pass 1 — Base Data

Business Context & SLA: High-criticality payment service. SLA = 99.9% availability.

Current Configuration: Compute: 6 m5.large. Network topology: basic_vpc_peering (cross-region peering for an external payment provider integration).

Key Metrics: Network p95_latency_ms > 170 during business hours (40–70 otherwise). error_rate 0.005–0.015 during business hours, 0.001–0.004 otherwise. Compute application_p95_latency_ms baseline 180–240, rising to 550–680 during business hours.

Time Pattern: Weekday business hours. Weekend levels are at baseline.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: Elevated network latency during business hours.

Target Recommendation: Optimize inter-service communication, add retries with exponential backoff, and switch from basic VPC peering to PrivateLink for the payment provider integration. Expected outcome: network p95 latency reduction from 170ms to below 80ms.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): When network p95_latency_ms > 170, compute application_p95_latency_ms increases from its Pass 1 baseline (180–240) to above 550ms within the same 15-min window. Pattern repeats on at least 11 of 14 days.

Key Evidence (with before/after inlined): Strong correlation between network and application latency. In other apps with identical basic VPC peering configuration, after switching to PrivateLink and adding retries with backoff, network p95 latency fell below 80ms and application p95 latency fell below 220ms while SLA stayed at 99.9%.

Scenario Metadata Sidecar

Compute Instance Class: m5.large

Compute Instance Count: 6

Network Topology: basic_vpc_peering

Monthly Cost Usd: 4100

Before After: config: basic_vpc_peering, no retry policy → change: PrivateLink + retries with exponential backoff → outcome: network p95 below 80ms, app p95 below 220ms, SLA 99.9%

Scenario 11: Multi-Tier Over-provisioning

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Evaluation properties: *Cross-tier correlation reading · Cost-axis reasoning · Mixed-state handling*

Pass 1 — Base Data

Business Context & SLA: Medium-criticality internal analytics platform. SLA = 99.7% availability.

Current Configuration: Compute: 12 m5.2xlarge. Database: db.r6g.4xlarge with 1 read replica. Network: standard cross-AZ. No scaling policies anywhere.

Key Metrics: All tiers below 30% utilization consistently. Compute cpu_p50 14–22%, cpu_p95 22–28%, memory_p50 18–26%. Database cpu_p90 14–22%, connections_p95 25–55. Network p95_latency_ms 35–65, error_rate 0.001–0.004.

Time Pattern: Consistently low utilization with minor weekday/weekend variation only.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: All three tiers heavily over-provisioned.

Target Recommendation: Comprehensive right-sizing exercise: compute m5.2xlarge × 12 → m5.large × 6, database db.r6g.4xlarge → db.r6g.xlarge. Projected total monthly savings: ~\$4,200.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): All three tiers simultaneously show low utilization (below 30%) consistently across all 14 days.

Key Evidence (with before/after inlined): All three tiers heavily over-provisioned. In other apps with identical fixed-large-instance configuration, after comprehensive right-sizing, all tiers stayed below 30% utilization and SLA remained at 99.7% with approximately \$4,200 monthly savings.

Scenario Metadata Sidecar

Compute Instance Class: m5.2xlarge

Compute Instance Count: 12

Compute Scaling Policy: none

Database Instance Class: db.r6g.4xlarge

Database Read Replicas: 1

Monthly Cost Usd: 11800

Before After: config: m5.2xlarge × 12 + db.r6g.4xlarge fixed → change: m5.large × 6 + db.r6g.xlarge → outcome: all tiers stayed below 30% utilization, SLA 99.7%, ~\$4,200/month savings

Scenario 12: Healthy Compute, Problematic Database

Type: Mixed

Tiers used: Compute_Metrics, Database_Metrics.

Evaluation properties: *Mixed-state handling · Single-tier action under mixed signals · Cost-axis reasoning*

Pass 1 — Base Data

Business Context & SLA: High-criticality user profile service. SLA = 99.9% availability.

Current Configuration: Compute: 6 m5.large with basic scaling (healthy, no change needed). Database: db.r6g.2xlarge with 1 read replica (oversized).

Key Metrics: Compute uses Healthy Baselines (section 5.2): cpu_p50 48–62%, cpu_p95 68–78%, memory_p50 50–62%, application_p95_latency_ms 140–200. Database: cpu_p90 < 23%, connections_p95 20–40, slow_query_percent 0.5–1.2%, db_query_p95_latency_ms 40–80.

Time Pattern: Normal weekday business pattern on compute; flat low database utilization across all 14 days.

Cross-Tier Correlation: None.

Key Evidence: Clear mismatch between healthy compute (in healthy ranges) and over-provisioned database (consistently under-utilized).

Target Recommendation: Compute tier is correctly sized — do not touch. Downsize database instance from db.r6g.2xlarge to db.r6g.large while leaving compute and read replicas unchanged. Expected savings: ~\$1,400/month.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Healthy compute metrics, over-provisioned database. In other apps with identical compute/database split, downsizing only the database produced the same healthy compute metrics and SLA.

Scenario Metadata Sidecar

Compute Instance Class: m5.large

Compute Instance Count: 6

Compute Scaling Policy: basic

Database Instance Class: db.r6g.2xlarge

Database Read Replicas: 1

Monthly Cost Usd: 7200

Before After: config: m5.large × 6 healthy + db.r6g.2xlarge oversized → change: downsize DB to db.r6g.large only → outcome: compute metrics unchanged, SLA 99.9%, ~\$1,400/month savings

Scenario 13: Compute Spike + Database Strain

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Database_Metrics.

Evaluation properties: *Cross-tier correlation reading · Trade-off reasoning · Time-pattern recognition*

Pass 1 — Base Data

Business Context & SLA: High-criticality search service. SLA = 99.9% availability.

Current Configuration: Compute: 8 m5.large with basic scaling (min 8, max 12). Database: db.r6g.xlarge with 0 read replicas, connection pool = 150.

Key Metrics: Compute baseline cpu_p50 32–42%, p90 50–62%. During peak-hour spikes: cpu_p95 86–94%. Memory_p50 40–55%, p90 55–72%. application_p95_latency_ms 180–260 baseline, 420–580 during spikes. Database baseline cpu_p90 35–45%, connections_p95 40–70. During correlated peaks: cpu_p90 65–80%, connections_p95 130–160. slow_query_percent 1.5–3% baseline, rising to 6–9% during peaks.

Time Pattern: Weekday peak-hour spikes (11 AM and 3 PM). Weekends baseline only.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: Compute spikes and elevated database connections during weekday peaks.

Target Recommendation: Implement predictive auto-scaling on compute tier (scale out at cpu_p90 > 65%), add 2 read replicas with read/write splitting on the database, and raise connection pool limit from 150 to 300.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): During compute cpu_p95 > 85%, database connections_p95 rises by 3x above its Pass 1 baseline (from 40–70 to 130–210) within the next 15-min window. Pattern repeats during peak hours on at least 11 of 14 weekday peak windows.

Key Evidence (with before/after inlined): Strong correlation between compute CPU spikes and database connection count. In other apps with identical compute/database configuration, after predictive auto-scaling and adding 2 read replicas with pool=300, spikes were absorbed without incident and SLA stayed at 99.9%.

Scenario Metadata Sidecar

Compute Instance Class: m5.large

Compute Instance Count: 8

Compute Scaling Policy: basic (min 8, max 12)

Database Instance Class: db.r6g.xlarge

Database Read Replicas: 0

Database Connection Pool Limit: 150

Monthly Cost Usd: 7800

Before After: config: m5.large × 8 basic scaling + db.r6g.xlarge no replicas, pool=150 → change: predictive scaling on compute + 2 read replicas + pool=300 → outcome: spikes absorbed, SLA 99.9% preserved

Scenario 14: Good Performance, High Cost

Type: Mixed

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Evaluation properties: *Mixed-state handling · Cost-axis reasoning · Restraint on performance*

Pass 1 — Base Data

Business Context & SLA: High-criticality checkout flow. SLA = 99.9% availability.

Current Configuration: Compute: 12 m5.2xlarge fixed. Database: db.r6g.4xlarge with 2 read replicas. Network: PrivateLink.

Key Metrics: Compute: cpu_p50 18–28%, cpu_p95 28–38%, memory_p50 22–32%, application_p95_latency_ms 110–170 (excellent). Database: cpu_p90 18–28%, connections_p95 30–55, slow_query_percent 0.4–1.2%, db_query_p95_latency_ms 35–75. Network: p95_latency_ms 25–50, error_rate 0.001–0.005.

Time Pattern: Stable healthy weekday business pattern, weekend baseline ~50%.

Cross-Tier Correlation: None.

Key Evidence: Excellent application and database latency, near-zero errors, but utilization is well below healthy targets across the board — over-provisioned for current load.

Target Recommendation: Application performance is excellent and must be preserved. Right-size compute (m5.2xlarge × 12 → m5.large × 8) and database (db.r6g.4xlarge → db.r6g.xlarge) while leaving the read replicas and network unchanged. Expected outcome: monthly cost down approximately 38% (~\$5,500 saved) with no degradation in application or network latency.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Excellent performance metrics with poor cost efficiency. In other apps with identical oversized configuration, after right-sizing, application p95 latency stayed in the 110–180ms range and monthly cost dropped approximately 38%.

Scenario Metadata Sidecar

Compute Instance Class: m5.2xlarge

Compute Instance Count: 12

Compute Scaling Policy: none

Database Instance Class: db.r6g.4xlarge

Database Read Replicas: 2

Network Topology: private_link

Monthly Cost Usd: 14400

Before After: config: m5.2xlarge × 12 + db.r6g.4xlarge → change: m5.large × 8 + db.r6g.xlarge (replicas + network unchanged) → outcome: app p95 latency unchanged at 110–180ms, monthly cost down ~38% (~\$5,500/month)

Scenario 15: Reliability Focused Over-provisioning

Type: Mixed

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Evaluation properties: *SLA-aware reasoning · Trade-off reasoning · Business-context questioning*

Pass 1 — Base Data

Business Context & SLA: High-criticality payment processing. SLA = 99.99% availability (current).

Current Configuration: Compute: 16 m5.xlarge fixed. Database: db.r6g.2xlarge with 3 read replicas in 3 AZs. Network: PrivateLink across 3 AZs (over-provisioned for redundancy).

Key Metrics: Compute cpu_p50 18–28%, cpu_p95 28–34%, memory_p50 22–32%, application_p95_latency_ms 90–150 (excellent). Database cpu_p90 22–32%, connections_p95 30–60, slow_query_percent 0.3–0.9%, db_query_p95_latency_ms 30–70. Network p95_latency_ms 20–45, error_rate < 0.0001 (extremely low).

Time Pattern: Stable but wasteful resource usage across all 14 days. No meaningful weekday/weekend variation.

Cross-Tier Correlation: None at Pass 1.

Key Evidence: Extremely low error rate at the expense of very high cost. All tiers significantly under-utilized.

Target Recommendation: System achieves extremely low error rate through heavy over-provisioning. The right next step is a business-context conversation: is the 99.99% SLA actually required by the business, or is 99.95% acceptable? If 99.95% is acceptable, right-sizing all tiers can reduce monthly cost by approximately 45% (~\$8,200 saved) while still preserving an error rate below 0.01%.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): All tiers show utilization below 35% while error rate remains below 0.01% throughout the 14-day window.

Key Evidence (with before/after inlined): Extremely low error rate at the expense of very high cost. In other apps with identical fixed-large configuration, after SLA review and targeted right-sizing to a 99.95% target, error rate stayed below 0.01% and monthly cost dropped significantly.

Scenario Metadata Sidecar

Compute Instance Class: m5.xlarge

Compute Instance Count: 16

Compute Scaling Policy: none

Database Instance Class: db.r6g.2xlarge

Database Read Replicas: 3

Network Topology: private_link_multi_az

Monthly Cost Usd: 18200

Before After: config: m5.xlarge × 16 + db.r6g.2xlarge + 3 replicas + multi-AZ PrivateLink (99.99% SLA) → change: review business need for 99.99%; if 99.95% acceptable, right-size all tiers → outcome: error rate stayed below 0.01%, ~\$8,200/month savings

Scenario 16: Partial Optimization

Type: Single-tier (Mild Negative)

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Evaluation properties: *Restraint · Single-tier detection · Mixed-state handling*

Pass 1 — Base Data

Business Context & SLA: Medium-criticality reporting dashboard. SLA = 99.7% availability.

Current Configuration: Compute: 4 m5.large fixed (slightly oversized). Database: db.r6g.large with 1 read replica (healthy). Network: standard (healthy).

Key Metrics: Compute: cpu_p50 22–32%, cpu_p95 38–48%, memory_p50 28–38%, application_p95_latency_ms 120–180 (healthy by latency, mildly low on utilization). Database uses Healthy Baselines (cpu_p90 38–48%, connections_p95 45–75, slow_query_percent 0.8–1.8%, db_query_p95_latency_ms 50–90). Network uses Healthy Baselines (p95_latency_ms 30–60, error_rate 0.002–0.006).

Time Pattern: Mildly low but stable utilization on compute; weekday business pattern on others.

Cross-Tier Correlation: None.

Key Evidence: Only compute tier shows mild underutilization; other tiers in healthy range.

Target Recommendation: Compute tier shows mild underutilization but is well above the threshold that would justify aggressive right-sizing. Recommend a targeted single-step adjustment: reduce compute fleet from 4 to 3 m5.large instances. Do not change instance class. Leave database and network alone. Expected savings: ~\$320/month.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Only compute tier shows mild underutilization. In other apps with identical compute configuration, the targeted right-sizing (4 → 3 m5.large) reduced cost with no SLA impact.

Scenario Metadata Sidecar

Compute Instance Class: m5.large

Compute Instance Count: 4

Compute Scaling Policy: none

Database Instance Class: db.r6g.large

Database Read Replicas: 1

Monthly Cost Usd: 2400

Before After: config: m5.large × 4 fixed → change: m5.large × 3 fixed → outcome: cost down ~\$320/month, SLA 99.7% preserved

Scenario 17: Cross-Tier Performance Degradation

Type: Cross-tier (Negative)

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Evaluation properties: *Diagnostic deferral · Cross-tier correlation reading · Restraint under ambiguity*

Pass 1 — Base Data

Business Context & SLA: High-criticality core API platform. SLA = 99.9% availability.

Current Configuration: No optimization across tiers. Compute: 10 m5.large basic scaling. Database: db.r6g.xlarge with 1 read replica. Network: basic VPC peering.

Key Metrics: During peak hours, all three latency signals are elevated simultaneously with no clear lead-lag. Compute application_p95_latency_ms 380–560. Database db_query_p95_latency_ms 220–380. Network p95_latency_ms 140–210. CPU and connection counts are within normal ranges across all tiers — the problem is latency-only, not capacity.

Time Pattern: Latency rises across all tiers during weekday peak hours (10 AM – 4 PM).

Cross-Tier Correlation: None at Pass 1.

Key Evidence: High latency across all three tiers, but no clear evidence of which tier is the root cause — all three rise together within the same 15-min windows.

Target Recommendation: Performance degradation is observed across all three tiers simultaneously during peak hours. Capacity metrics are normal; the problem is latency-distributed, not capacity-driven. Recommend full end-to-end distributed trace analysis to identify the actual root cause (which could be a downstream provider, a shared queue, an external dependency, or coordination overhead) before making any scaling or right-sizing decisions. Aggressive action without diagnosis risks worsening the problem.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): Latency increases simultaneously in network, database, and compute tiers during peak hours on at least 11 of 14 weekday peak windows. All three signals cross their elevated thresholds within the same 15-min window — neither leads nor lags meaningfully.

Key Evidence (with before/after inlined): Correlated latency spikes across all three tiers with no clear lead-lag relationship. In other apps with identical no-optimization configuration, after full end-to-end trace analysis identified the actual root cause (which varied across cases), targeted intervention reduced latency significantly while SLA stayed at 99.9%.

Scenario Metadata Sidecar

Compute Instance Class: m5.large

Compute Instance Count: 10

Compute Scaling Policy: basic

Database Instance Class: db.r6g.xlarge

Database Read Replicas: 1

Network Topology: basic_vpc_peering

Monthly Cost Usd: 9800

Before After: config: no optimization across tiers → change: end-to-end distributed trace analysis identified root cause (varies across cases), targeted intervention applied → outcome: latency reduced significantly, SLA 99.9% preserved

Scenario 18: Mostly Healthy with Minor Inefficiency

Type: Mostly Healthy

Tiers used: Compute_Metrics, Database_Metrics, Network_Metrics.

Pass 1 — Base Data

Business Context & SLA: Medium-criticality internal tool. SLA = 99.5% availability.

Current Configuration: Compute: 5 t3.medium fixed (slightly oversized). Database: db.r6g.large with 1 read replica (healthy). Network: standard (healthy).

Key Metrics: Compute: cpu_p50 32–42%, cpu_p95 50–60%, memory_p50 38–48%, application_p95_latency_ms 130–190 (healthy on latency, slightly low on utilization). Database uses Healthy Baselines (cpu_p90 38–50%, connections_p95 45–70, slow_query_percent 0.8–2.0%, db_query_p95_latency_ms 45–85). Network uses Healthy Baselines.

Time Pattern: Normal weekday business pattern, weekend ~50% of weekday baseline.

Cross-Tier Correlation: None.

Key Evidence: Most metrics in healthy range. Compute slightly below ideal utilization but well above the threshold that would justify aggressive change. Border case between fully healthy and partial-optimization.

Target Recommendation: System is mostly well-optimized. Recommend a minor compute right-sizing adjustment: reduce compute fleet from 5 to 4 t3.medium instances. No other changes. Expected savings: ~\$110/month.

Pass 2 — Correlation and Inlined Evidence

Cross-Tier Correlation (Pass 2): None — Pass 2 is pass-through for time-series.

Key Evidence (with before/after inlined): Mostly healthy metrics with only small room for improvement. In other apps with identical mostly-optimal configuration, the minor compute right-sizing (5 → 4) reduced cost with no SLA impact.

Scenario Metadata Sidecar

Compute Instance Class: t3.medium

Compute Instance Count: 5

Compute Scaling Policy: none

Database Instance Class: db.r6g.large

Database Read Replicas: 1

Monthly Cost Usd: 1900

Before After: config: t3.medium × 5 fixed → change: t3.medium × 4 fixed → outcome: cost down ~\$110/month, SLA 99.5% preserved

7. LLM Prompt Scaffolds

Two prompt templates, one for each pass. Both are designed to be filled in programmatically per scenario. Suggested temperature: 0.2–0.4 for reproducibility. The `scenario_metadata` sidecar is authored statically per scenario and passed through verbatim by both passes — no LLM generation is needed for that block.

7.1 Pass 1 prompt template

```
SYSTEM:
You are a synthetic-data generator for cloud telemetry. You produce realistic
time-series metrics that match a specified scenario exactly. You output ONLY
valid JSON – no prose, no markdown fencing, no explanation.

USER:
Generate Pass 1 (base time-series) for the following cloud governance scenario.

SCENARIO {scenario_num}: {scenario_name}
Type: {scenario_type}
Tiers required: {tiers_used}

Business Context & SLA: {p1_business}
Current Configuration: {p1_config}
Key Metrics (must hold across the dataset): {p1_metrics}
Time Pattern: {p1_pattern}
Cross-Tier Correlation: None at this pass – generate each tier independently.
Key Evidence the data must show: {p1_evidence}

REQUIREMENTS:
- 15-minute intervals starting at 2026-05-01T00:00:00Z (a Monday)
- Exactly 1,344 records per tier (14 days × 96 intervals/day)
- Aligned timestamps across all tier arrays
- Realistic daily patterns: business-hours behavior on weekdays;
weekend behavior per section 1.8 (typically 40-60% of weekday baseline
for production-facing; 10-25% for internal services; healthy services
retain their normal range)
- Values must remain within the ranges stated in Key Metrics
- For "healthy" tiers, use the Healthy Baselines from section 5.2
- ISO 8601 UTC timestamps
- For tiers not in "Tiers required", output an empty array
- Emit the scenario_metadata sidecar block verbatim (provided below)

SCENARIO METADATA (emit verbatim):
{scenario_metadata_json}

OUTPUT SCHEMA (strict):
{{
  "scenario_id": {scenario_num},
  "scenario_name": "{scenario_name}",
  "pass": 1,
  "scenario_metadata": <emit verbatim from above>,
  "Compute_Metrics": [
    {{ "timestamp": "...", "cpu_p50": 0.0, "cpu_p90": 0.0,
      "cpu_p95": 0.0, "memory_p50": 0.0, "memory_p90": 0.0,
      "application_p95_latency_ms": 0.0 }}
  ],
  "Database_Metrics": [
    {{ "timestamp": "...", "cpu_p90": 0.0, "connections_p95": 0,
      "slow_query_percent": 0.0, "db_query_p95_latency_ms": 0.0 }}
  ],
```



```

"Network_Metrics": [
  {"timestamp": "...", "p95_latency_ms": 0.0, "error_rate": 0.0}
],
"Cache_Metrics": [
  {"timestamp": "...", "hit_ratio": 0.0}
]
}]

```

Output the JSON only.

7.2 Pass 2 prompt template

SYSTEM:

You are a synthetic-data refiner. You take an existing Pass 1 JSON output and surgically add cross-tier correlations as specified, while preserving every Pass 1 signal exactly. You output ONLY valid JSON – no prose, no markdown fencing, no explanation.

USER:

Refine the following Pass 1 output for Scenario {scenario_num}.

SCENARIO {scenario_num}: {scenario_name}

Type: {scenario_type}

PASS 1 INPUT (do not modify outside the correlation rule):

{pass1_json}

PASS 2 INSTRUCTION:

Cross-Tier Correlation to apply: {p2_correlation}

Key Evidence (inlined before/after, emit in scenario_metadata): {p2_evidence}

BASELINE REFERENCE:

Correlation magnitudes (e.g. "+38-45%", "3x") are RELATIVE to the Pass 1 baseline ranges for the affected tier. Compute absolute target values by applying the relative bump to the Pass 1 baseline. Do not treat the correlation magnitude as an absolute value.

REQUIREMENTS:

- If Cross-Tier Correlation is "None", output the Pass 1 JSON UNCHANGED (with "pass" updated to 2).
- If a correlation is specified, modify only the dependent tier's values in the time windows where the source signal exceeds its threshold.
- Apply correlation adjustments within the stated timing window (e.g. "same 15-min window" or "within 15-30 minutes").
- The pattern must hold on at least 11 of 14 days, or all 10 weekdays for weekday-only patterns.
- DO NOT alter the source signal. DO NOT alter timestamps.
- DO NOT regenerate base values. Only add the correlation effect on top.
- Preserve all values in tiers not involved in the correlation.
- Preserve the scenario_metadata sidecar exactly as received.

OUTPUT SCHEMA (strict):

```

{{
  "scenario_id": {scenario_num},
  "scenario_name": "{scenario_name}",
  "pass": 2,
  "scenario_metadata": <unchanged from Pass 1>,
  "Compute_Metrics": [...],
  "Database_Metrics": [...],

```

```
"Network_Metrics": [...],  
"Cache_Metrics": [...]  
}}
```

Output the JSON only.

7.3 Chunked generation (if context-limited)

1,344 records per tier across up to four tiers can exceed practical LLM output budgets for some models. If the chosen model cannot emit a full scenario in one response, generate in day-chunks (96 records per tier per call) and concatenate in post-processing. Each chunked call includes the scenario context, the chunk's date range, and an instruction to maintain continuity with the previous chunk's tail values. Continuity is verified deterministically by the validator, not by the model.

8. Quality Assurance

8.1 Deterministic checks

These checks are programmatic and do not require the downstream agent system. Every generated scenario should pass all of them before being accepted into the dataset.

Check	Rule	Failure mode caught
Record count	Each non-empty tier array contains exactly 1,344 records.	Truncated generation.
Timestamp continuity	Timestamps are 15 minutes apart, strictly monotonic increasing, no gaps or duplicates, starting at 2026-05-01T00:00:00Z (Monday).	Drift, skipped intervals, duplicates.
Cross-tier alignment	Timestamps match index-by-index across all non-empty tier arrays.	Misaligned tiers — fatal for correlation analysis.
Value range conformance	All numeric fields fall within the scenario's stated ranges, with tolerance bands.	Out-of-spec values; LLM hallucinated metric ranges.
Schema conformance	JSON validates against the v2 schemas including new fields (application_p95_latency_ms, db_query_p95_latency_ms).	Missing fields; wrong types.
Sidecar presence	scenario_metadata block present and matches the scenario's authored sidecar exactly.	Sidecar dropped or modified.
Pattern frequency	Recurring patterns hold on at least 11 of 14 days, or all 10 weekdays for weekday-only patterns.	Insufficient pattern strength.
Weekend behavior	Weekend traffic per section 1.8 — falls within the stated ratio of weekday baseline.	Weekday pattern bleeding into weekends, or zero-weekend for production services.
Pass 2 invariance	For tiers not affected by the Pass 2 correlation rule, Pass 2 values match Pass 1 values exactly. For correlation-free scenarios, the entire Pass 2 time-series matches Pass 1.	Pass 2 regenerated base data.
Correlation timing	Where Pass 2 specifies a timing window, the dependent tier's adjustment falls within that window of the source signal crossing its threshold.	Loose correlations; missed cause-effect coupling.
Correlation magnitude (relative)	Where Pass 2 specifies a relative magnitude, measured adjustments correspond to the magnitude applied to the Pass 1 baseline (per section 1.9).	Treated relative magnitude as absolute value.
No spurious correlation	For scenarios marked Cross-Tier Correlation: None, statistical correlation between tier signals stays below a chosen noise threshold.	Spurious correlations baked into base data.
Sidecar field presence (new in v3)	Sidecar fields referenced by the §2.6 read contract are present and non-empty for any scenario whose target recommendation depends on them (e.g. top_queries for scenarios 4 and 8, top_cache_keys for scenario 7, per_instance_imbalance for scenario 5).	Recommendation cites sidecar fixture that was never authored; agent reasoning hits a NoneType.

8.2 Deferred to the agent system

Some quality properties can only be evaluated by running the agent system that consumes the dataset. These should be acknowledged as deferred QA and revisited once the agent system is operational:

- Whether the evidence in a scenario is sufficient for an LLM to arrive at the target recommendation without prompting tricks.
- Whether the recommendation produced by the agent matches the scenario's target recommendation in substance, including the specific quantities sourced from the sidecar.
- Whether the correlation strength is detectable by the Cross-Tier Evaluator without being so strong that it makes the problem trivial.
- Whether the restraint scenarios (6, 16, 18) successfully resist over-recommendation.
- Whether the diagnostic deferral scenario (17) successfully induces the agent to recommend further investigation.
- Whether the business-context questioning scenario (15) elicits an SLA-review recommendation rather than premature right-sizing.

8.3 Validator sketch

```
# qa_validator.py – sketch
from datetime import datetime, timedelta

EXPECTED_RECORDS = 1344
START_TS = "2026-05-01T00:00:00Z" # Monday
INTERVAL_MIN = 15

def validate_record_count(tier_array, name):
    assert len(tier_array) == EXPECTED_RECORDS, \
        f"{name}: got {len(tier_array)} records, expected {EXPECTED_RECORDS}"

def validate_timestamp_continuity(tier_array, name):
    expected = datetime.fromisoformat(START_TS.replace("Z", "+00:00"))
    for i, rec in enumerate(tier_array):
        actual = datetime.fromisoformat(rec["timestamp"].replace("Z", "+00:00"))
        assert actual == expected, f"{name}[{i}]: ts mismatch"
        expected += timedelta(minutes=INTERVAL_MIN)

def validate_cross_tier_alignment(scenario_json):
    tiers = [t for t in ("Compute_Metrics", "Database_Metrics",
        "Network_Metrics", "Cache_Metrics")]
    if scenario_json.get(t):
        if len(tiers) < 2: return
        base = [r["timestamp"] for r in scenario_json[tiers[0]]]
        for t in tiers[1:]:
            assert [r["timestamp"] for r in scenario_json[t]] == base, \
                f"{t} timestamps misaligned with {tiers[0]}"

def validate_sidecar(scenario_json, expected_sidecar):
    actual = scenario_json.get("scenario_metadata")
    assert actual == expected_sidecar, \
        "scenario_metadata sidecar dropped or modified"

def validate_pass2_invariance(pass1_json, pass2_json, affected_tiers):
    for tier in ("Compute_Metrics", "Database_Metrics",
        "Network_Metrics", "Cache_Metrics"):
        if tier in affected_tiers: continue
        assert pass1_json.get(tier) == pass2_json.get(tier), \
            f"{tier} mutated in Pass 2 despite not being correlation-affected"

# Add: value-range checks (with healthy-baseline overrides),
```

```
# pattern-frequency checks (with weekday-only support),  
# weekend-ratio checks, correlation-timing checks,  
# correlation-magnitude checks (relative to Pass 1 baseline),  
# new field checks (application_p95_latency_ms, db_query_p95_latency_ms).
```

9. Deliverables and Suggested File Layout

```
dataset/
  ■■■ scenarios/
  ■ ■■■ manifest.json
  ■ ■■■ 01_chronic_underutilization/
  ■ ■ ■■■ scenario_spec.yaml # Source-of-truth (Pass 1 + Pass 2 + sidecar)
  ■ ■ ■■■ pass1.json # Pass 1 output
  ■ ■ ■■■ pass2.json # Pass 2 output (final)
  ■ ■ ■■■ qa_report.json # Validator output
  ■ ■■■ 02_spiky_compute_load/
  ■ ■ ■■■ ...
  ■ ...
  ■ ■■■ 18_mostly_healthy/
  ■ ■■■ ...
  ■■■ generation/
  ■ ■■■ prompts/
  ■ ■ ■■■ pass1_template.txt # From section 7.1
  ■ ■ ■■■ pass2_template.txt # From section 7.2
  ■ ■■■ generate.py # Driver: scenario_spec → LLM → JSON
  ■ ■■■ llm_client.py
  ■■■ qa/
  ■ ■■■ validator.py # Section 8.3 checks
  ■ ■■■ correlation_metrics.py # Timing, magnitude, frequency
  ■ ■■■ reports/
  ■■■ docs/
  ■ ■■■ healthy_baselines.md # Section 5.2 reference
  ■■■ README.md
```

10. Change Log

Version	Date	Change
v1	2026-05-24	Initial release. 18 scenarios, two-pass generation, basic schema, prompt scaffolds, QA section.
v2	2026-05-24	Adversarial-review patches: scenario_metadata sidecar, application_p95_latency_ms and db_query_p95_latency_ms fields, Healthy Baselines appendix, recommendation-sourcing principle, weekday/weekend convention, Pass 2 baseline reference, inlined before/after evidence into Pass 2 Key Evidence, explicit instance classes and counts across all scenarios, quantitative fill-ins for scenarios 13/14/18.
v3	2026-05-24	Agent-design feedback: consumer-side data access contract (§2.6), bundle layout (§2.7), tightened recommendation-sourcing principle with read-operation mapping, new QA check for sidecar-field presence against the read contract.

End of document.